

Lesson 10 Structured Note: Parenthesis Matching Implementation with an Array Char Stack

0. Current Position in the Course

You are now in **Lesson 10** of the stack curriculum.

Before this lesson, you learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO : Last In, First Out.
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations and helper functions.
Lesson 7	Complete C linked-list stack implementation.
Lesson 8	C++ class implementation of a linked-list stack.
Lesson 9	Parenthesis matching as a stack application.

Lesson 9 taught the **logic** of parenthesis matching.

Lesson 10 now teaches the **actual implementation** using an **array stack that stores characters**.

1. Lesson 10 Goal

By the end of this lesson, you should understand:

- Why parenthesis matching uses a stack.
 - Why the stack stores `char` values instead of `int` values.
 - How a string is scanned character by character.
 - What the null terminator `\0` means.
 - Why `strlen(exp)` is used as the stack size.
 - How to define an array stack for characters.
 - How to push opening parentheses `(`.
 - How to handle closing parentheses `)`.
 - How to detect an extra closing parenthesis.
 - How to detect an extra opening parenthesis.
 - Why all non-parenthesis characters are ignored.
 - How to write the full `isBalanced()` function.
 - How to dry-run the algorithm.
 - The time and space complexity of the solution.
-

2. Quick Review: What Problem Are We Solving?

We want to check whether the parentheses in an expression are **balanced**.

Balanced means:

- Every opening parenthesis `(` has a matching closing parenthesis `)`.
- Every closing parenthesis `)` has a previous unmatched opening parenthesis `(`.
- Parentheses close in the correct order.
- No opening parenthesis is left over at the end.
- No closing parenthesis appears too early.

Example of a balanced expression:

```
((A+B))
```

This is balanced because every `(` gets matched with a `)`.

Example of an unbalanced expression:

```
((A+B)
```

This is not balanced because one opening parenthesis is not closed.

Another unbalanced expression:

```
(A+B))
```

This is not balanced because there is one extra closing parenthesis.

3. Why a Stack Is Useful Here

A stack follows this rule:

```
Last In, First Out
```

This is also called:

```
LIFO
```

Parentheses also follow this idea.

Look at this expression:

```
((A+B))
```

The second `(` is opened after the first `(`.

So the second `(` must close before the first `(` closes.

That is LIFO behavior:

```
Last opened parenthesis must be closed first.
```

That is why a stack is a good tool for parenthesis matching.

4. Main Rule of Parenthesis Matching

When scanning the expression from left to right:

Character found	Action
(	Push it onto the stack.
)	Pop one (from the stack.
Anything else	Ignore it.

Simple meaning:

Opening parenthesis waits for a match.
Closing parenthesis cancels one previous opening parenthesis.

5. What Changed from the Old Array Stack?

Earlier, the array stack stored integers.

The old stack used:

```
int *S;
```

That means the stack stored integer values like:

10, 20, 30

But in parenthesis matching, we are storing characters.

Parentheses are characters:

```
'('
')'
```

So now the stack should use:

```
char *S;
```

That means the stack stores character values.

The stack rule does not change.

Only the stored data type changes.

6. Array Char Stack Structure

For Lesson 10, the stack structure is:

```
struct Stack {  
    int size;  
    int top;  
    char *S;  
};
```

Meaning of each field:

Field	Simple meaning
size	Maximum number of characters the stack can store.
top	Index of the current top character.
S	Pointer to the character array used as the stack.

Important point:

top stores an index, not the actual character.

Example:

```
top = 1;  
S[top] = '(';
```

This means the top element is stored at index 1.

7. Empty and Full Conditions

The stack is still an array stack, so the same empty and full formulas apply.

Empty Condition

The stack is empty when:

```
top == -1
```

This means there is no active element in the stack.

Important:

```
top = -1 does not mean S[-1] exists.
```

Array index `-1` is invalid.

It is only a marker meaning:

```
The stack is empty.
```

Full Condition

The stack is full when:

```
top == size - 1
```

Example:

```
size = 5;
```

Valid indices are:

```
0, 1, 2, 3, 4
```

The last valid index is:

```
size - 1 = 4
```

So the stack is full when:

```
top == 4
```

8. What Is a String in C?

An expression like this:

```
"((A+B))"
```

is a string.

In C, a string is a character array that ends with a special character:

```
'\0'
```

This special character is called the **null terminator**.

It marks the end of the string.

Example:

```
"((A+B))"
```

is stored like this:

```
'(' '(' 'A' '+' 'B' ')' ')' '\0'
```

The `\0` is not part of the expression itself.

It only tells the program:

```
The string ends here.
```

9. Why the Loop Checks `exp[i] != '\0'`

To scan the string, we use:

```
for (int i = 0; exp[i] != '\0'; i++)
```

This means:

```
Start at index 0.  
Keep moving forward until the end of the string.  
Stop when the null terminator is reached.
```

The null terminator is not pushed onto the stack.

It only controls when the loop stops.

10. Why Use `strlen(exp)` as Stack Size?

The function `strlen(exp)` gives the number of characters in the expression, not counting `\0`.

Example:

```
char exp[] = "((A+B))";
```

The visible characters are:

```
( ( A + B ) )
```

There are 7 visible characters.

So:

```
strlen(exp)
```

returns:

```
7
```

Why is this a safe stack size?

Worst case, every character could be an opening parenthesis.

Example:


```
"(((((((
```

If every character is `(`, we may need to push every character.

So the maximum number of characters the stack might hold is the expression length.

That is why this is safe:

```
st.size = strlen(exp);
```

Simple meaning:

Make the stack large enough to hold all possible opening parentheses.

11. Header Files Needed

For the C implementation, we need:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Meaning:

Header file	Why it is needed
<code>stdio.h</code>	For <code>printf()</code> .
<code>stdlib.h</code>	For <code>malloc()</code> and <code>free()</code> .
<code>string.h</code>	For <code>strlen()</code> .

12. Helper Function: `isEmpty()`

Meaning

`isEmpty()` checks whether the stack has no active elements.

Code:

```
int isEmpty(struct Stack st) {  
    return st.top == -1;  
}
```

Return values:

Return value	Meaning
1	Stack is empty.
0	Stack is not empty.

Example:

```
top = -1;
```

Then:

```
isEmpty(st)
```

returns:

```
1
```

because the stack is empty.

13. Helper Function: `isFull()`

Meaning

`isFull()` checks whether the stack has no free space left.

Code:

```
int isFull(struct Stack st) {  
    return st.top == st.size - 1;  
}
```

Return values:

Return value	Meaning
1	Stack is full.
0	Stack is not full.

Example:

```
size = 5;  
top = 4;
```

Check:

```
top == size - 1  
4 == 5 - 1  
4 == 4
```

This is true.

So the stack is full.

14. Push Operation for Character Stack

Meaning

`push()` inserts a character at the top of the stack.

For this lesson, we mainly push:

```
'('
```

Code:

```
void push(struct Stack *st, char x) {
    if (isFull(*st)) {
        printf("Stack Overflow\n");
    } else {
        st->top++;
        st->S[st->top] = x;
    }
}
```

Why does `push()` use `struct Stack *st`?

`push()` changes the real stack.

It changes:

```
st->top
st->S[st->top]
```

So it must receive the stack by pointer.

That is why the parameter is:

```
struct Stack *st
```

What happens during a successful push?

A successful push does two things:

1. Increase `top` by 1.
2. Store the new character at `S[top]`.

Example:

Before push:

```
top = -1
stack = empty
```

Call:

```
push(&st, '(');
```

Step 1:

```
top: -1 -> 0
```

Step 2:

```
S[0] = '('
```

Now the stack contains one opening parenthesis.

15. Pop Operation for Character Stack

Meaning

`pop()` removes and returns the top character from the stack.

Code:

```
char pop(struct Stack *st) {  
    char x = '\0';  
  
    if (isEmpty(*st)) {  
        printf("Stack Underflow\n");  
    } else {  
        x = st->S[st->top];  
        st->top--;  
    }  
  
    return x;  
}
```

Why does `pop()` use `struct Stack *st`?

`pop()` changes the stack because it decreases `top`.

So it must receive the stack by pointer.

What happens during a successful pop?

A successful pop does three things:

1. Save the character at `S[top]`.
2. Decrease `top` by 1.
3. Return the saved character.

Example:

Before pop:

```
S[0] = '('  
top = 0
```

Call:

```
pop(&st);
```

Step 1:

```
x = S[0]  
x = '('
```

Step 2:

```
top: 0 -> -1
```

Now the stack is empty.

16. The `isBalanced()` Function

The main function in Lesson 10 is:

```
int isBalanced(char *exp)
```

It receives an expression as a character array.

It returns:

Return value	Meaning
1	Parentheses are balanced.
0	Parentheses are not balanced.

17. Main Algorithm in Simple Words

To check whether an expression is balanced:

1. Create a character stack.
2. Set its size to `strlen(exp)`.
3. Set `top = -1`.
4. Scan the expression from left to right.
5. If the current character is `(`, push it.
6. If the current character is `)`, check whether the stack is empty.
7. If the stack is empty, return `0` immediately.
8. Otherwise, pop one opening parenthesis.
9. Ignore all other characters.
10. After the scan, check whether the stack is empty.
11. If the stack is empty, return `1`.
12. If the stack is not empty, return `0`.

Simple meaning:

Every closing parenthesis must cancel one previous opening parenthesis.

18. Complete `isBalanced()` Code

```
int isBalanced(char *exp) {
    struct Stack st;

    st.size = strlen(exp);
    st.top = -1;
    st.S = (char *)malloc(st.size * sizeof(char));

    if (st.S == NULL && st.size > 0) {
        printf("Memory allocation failed\n");
        return 0;
    }
}
```

```
for (int i = 0; exp[i] != '\0'; i++) {
    if (exp[i] == '(') {
        push(&st, exp[i]);
    } else if (exp[i] == ')') {
        if (isEmpty(st)) {
            free(st.S);
            return 0;
        }
        pop(&st);
    }
}

int result = isEmpty(st);

free(st.S);

return result;
}
```

19. Explanation of `isBalanced()` Step by Step

Step 1: Create a stack variable

```
struct Stack st;
```

This creates a stack named `st`.

Step 2: Set the stack size

```
st.size = strlen(exp);
```

This makes the stack large enough for the expression.

If the expression has 7 visible characters, the stack size becomes 7.

Step 3: Mark the stack as empty

```
st.top = -1;
```

This means the stack currently has no active elements.

Step 4: Allocate memory for the character array

```
st.S = (char *)malloc(st.size * sizeof(char));
```

This creates the character array on the heap.

Simple meaning:

```
Reserve enough memory to store the stack characters.
```

Step 5: Check whether memory allocation failed

```
if (st.S == NULL && st.size > 0) {  
    printf("Memory allocation failed\n");  
    return 0;  
}
```

If memory allocation fails, `malloc()` returns `NULL`.

This check prevents the program from using invalid memory.

Step 6: Scan every character

```
for (int i = 0; exp[i] != '\0'; i++)
```

This loop scans the expression from left to right.

It stops when it reaches the null terminator `\0`.

Step 7: Push opening parentheses

```
if (exp[i] == '(') {  
    push(&st, exp[i]);  
}
```

If the current character is `(`, push it onto the stack.

Simple meaning:

This opening parenthesis is waiting for a matching closing parenthesis.

Step 8: Handle closing parentheses

```
else if (exp[i] == ')') {  
    if (isEmpty(st)) {  
        free(st.S);  
        return 0;  
    }  
    pop(&st);  
}
```

If the current character is `)`, we need a matching previous `(`.

So first we check whether the stack is empty.

If the stack is empty, there is no opening parenthesis to match this closing parenthesis.

That means the expression is not balanced.

Example:

A+B)

There is a closing parenthesis, but no previous opening parenthesis.

So the function returns:

0

If the stack is not empty, we pop one (.

Simple meaning:

This closing parenthesis matched one opening parenthesis.

Step 9: Ignore all other characters

Characters like these are ignored:

A, B, C, D, numbers, +, -, *, /, spaces

Example:

(A+B)

The algorithm only cares about:

()

It ignores:

A + B

Step 10: Final check

```
int result = isEmpty(st);
```

At the end, the stack must be empty.

If the stack is empty, every opening parenthesis was matched.

If the stack is not empty, some opening parenthesis was never closed.

Example:

```
((A+B)
```

One `(` remains in the stack.

So the expression is not balanced.

Step 11: Free the stack memory

```
free(st.S);
```

The array was created using `malloc()`.

So it should be released using `free()`.

This prevents memory leaks.

Step 12: Return the result

```
return result;
```

If `result` is `1`, the expression is balanced.

If `result` is `0`, the expression is not balanced.

20. Complete C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Stack {
    int size;
    int top;
    char *S;
};
```

```

int isEmpty(struct Stack st) {
    return st.top == -1;
}

int isFull(struct Stack st) {
    return st.top == st.size - 1;
}

void push(struct Stack *st, char x) {
    if (isFull(*st)) {
        printf("Stack Overflow\n");
    } else {
        st->top++;
        st->S[st->top] = x;
    }
}

char pop(struct Stack *st) {
    char x = '\0';

    if (isEmpty(*st)) {
        printf("Stack Underflow\n");
    } else {
        x = st->S[st->top];
        st->top--;
    }

    return x;
}

int isBalanced(char *exp) {
    struct Stack st;

    st.size = strlen(exp);
    st.top = -1;
    st.S = (char *)malloc(st.size * sizeof(char));

    if (st.S == NULL && st.size > 0) {
        printf("Memory allocation failed\n");
        return 0;
    }

    for (int i = 0; exp[i] != '\0'; i++) {
        if (exp[i] == '(') {
            push(&st, exp[i]);
        } else if (exp[i] == ')') {
            if (isEmpty(st)) {

```

```
        free(st.S);
        return 0;
    }
    pop(&st);
}

int result = isEmpty(st);

free(st.S);

return result;
}

int main() {
    char exp[] = "((A+B))";

    if (isBalanced(exp)) {
        printf("Balanced\n");
    } else {
        printf("Not Balanced\n");
    }

    return 0;
}
```

21. Expected Output

For this expression:

```
((A+B))
```

The program prints:

```
Balanced
```

22. Dry Run 1: Balanced Expression

Expression:

((A+B))

Only parentheses matter:

(())

Initial stack:

empty

Trace:

Step	Character	Action	Stack after action
1	(	push	(
2	(	push	((
3	A	ignore	((
4	+	ignore	((
5	B	ignore	((
6	)	pop	(
7	)	pop	empty

At the end:

Stack is empty.

Result:

Balanced

Reason:

Every opening parenthesis got a matching closing parenthesis.

23. Dry Run 2: Extra Closing Parenthesis

Expression:

((()))

Only parentheses matter:

(()))

Initial stack:

empty

Trace:

Step	Character	Action	Stack after action
1	(	push	(
2	(	push	((
3	)	pop	(
4	)	pop	empty
5	)	stack is empty, cannot pop	error

At step 5, the algorithm sees).

It needs a matching (.

But the stack is empty.

So the result is:

Not Balanced

Reason:

There is an extra closing parenthesis.

This type of error is detected immediately during scanning.

24. Dry Run 3: Extra Opening Parenthesis

Expression:

((A+B)

Only parentheses matter:

(()

Initial stack:

empty

Trace:

Step	Character	Action	Stack after action
1	(	push	(
2	(	push	((
3	A	ignore	((
4	+	ignore	((
5	B	ignore	((
6	)	pop	(

At the end:

Stack still contains one '('.

So the result is:

Not Balanced

Reason:

One opening parenthesis was never closed.

This type of error is detected at the final stack check.

25. Dry Run 4: Expression with No Parentheses

Expression:

A+B*C

The algorithm scans each character:

Character	Action
A	ignore
+	ignore
B	ignore
*	ignore
C	ignore

No parentheses are pushed or popped.

At the end, the stack is empty.

Result:

Balanced

Why?

Because there are no unmatched parentheses.

26. Dry Run 5: Balanced Parentheses but Invalid Expression

Expression:

$(A+)*(B)$

Parentheses only:

$() ()$

The algorithm says:

Balanced

But the mathematical expression is invalid because $A+$ is missing an operand before $)$.

Important idea:

Parenthesis matching checks only parentheses.
It does not check whether the full expression is mathematically correct.

27. Two Main Error Cases

There are two main ways the expression can be unbalanced.

Error Case 1: Extra Closing Parenthesis

Example:

$(A+B))$

Problem:

A closing parenthesis appears when the stack is already empty.

How it is detected:

```
if (isEmpty(st)) {  
    return 0;  
}
```

This error is detected immediately.

Error Case 2: Extra Opening Parenthesis

Example:

```
((A+B)
```

Problem:

```
One opening parenthesis remains in the stack after scanning is finished.
```

How it is detected:

```
return isEmpty(st);
```

This error is detected at the end.

28. Why Check `isEmpty()` Before `pop()`?

When we see a closing parenthesis `)`, we want to pop one opening parenthesis `(`.

But popping from an empty stack is not allowed.

That is called:

```
Stack Underflow
```

Example:

```
A+B)
```

When the algorithm sees `)`, the stack is empty.

There is no matching `(`.

So we must check:

```
if (isEmpty(st))
```

before calling:

```
pop(&st);
```

This prevents underflow and correctly detects an extra closing parenthesis.

29. Why the Final Check Is Necessary

Suppose the expression is:

```
((A+B)
```

During scanning, there is no extra closing parenthesis.

So the algorithm does not fail early.

But after scanning, the stack still contains one `(`.

That means one opening parenthesis was never closed.

So the final check is necessary:

```
return isEmpty(st);
```

Meaning:

```
If the stack is empty, balanced.  
If the stack is not empty, not balanced.
```

30. What Characters Are Ignored?

In Lesson 10, we only care about:

```
'('
')'
```

Everything else is ignored.

Ignored characters include:

- Letters: A, B, C, D
- Numbers: 1, 2, 3
- Operators: +, -, *, /
- Spaces

Example:

```
(A + B) * (C - D)
```

The algorithm only sees:

```
( ) ( )
```

It ignores:

```
A + B * C - D
```

31. What This Algorithm Checks

This algorithm checks:

```
Are the round parentheses balanced?
```

It checks whether:

- Every (has a matching).
- Every) has a previous unmatched (.

- No (remains at the end.
- No) appears too early.

32. What This Algorithm Does Not Check

This algorithm does not check full expression validity.

Example:

(A+)*(B)

The parentheses are balanced:

() ()

So the algorithm returns:

Balanced

But the expression itself is mathematically invalid.

So do not confuse these two questions:

Question	Answered by Lesson 10 algorithm?
Are the parentheses balanced?	Yes
Is the full mathematical expression valid?	No

33. Time Complexity

The time complexity is:

$O(n)$

Why?

Because the algorithm scans the expression once from left to right.

If the expression has n characters, the loop checks n characters.

Each stack operation is constant time:

```
push = O(1)
pop = O(1)
isEmpty = O(1)
```

So the total time is:

$O(n)$

34. Space Complexity

The space complexity is:

$O(n)$

Why?

Because in the worst case, many characters may be pushed onto the stack.

Worst-case example:

(((((

Every character is an opening parenthesis.

So the stack may store up to n characters.

Therefore, space complexity is:

$O(n)$

35. Important Code Pattern to Memorize

The heart of Lesson 10 is:

```
if (exp[i] == '(') {  
    push(&st, exp[i]);  
} else if (exp[i] == ')') {  
    if (isEmpty(st)) {  
        return 0;  
    }  
    pop(&st);  
}
```

Simple meaning:

Push opening parentheses.
For closing parentheses, pop one opening parenthesis.
If there is nothing to pop, the expression is not balanced.

36. Common Beginner Mistakes

Mistake 1: Using `int *S` instead of `char *S`

Wrong for Lesson 10:

```
int *S;
```

Correct:

```
char *S;
```

Reason:

Parentheses are characters, not integers.

Mistake 2: Forgetting `#include <string.h>`

If you use:

```
strlen(exp)
```

then you need:

```
#include <string.h>
```

Mistake 3: Forgetting to set `top = -1`

Wrong:

```
st.size = strlen(exp);  
st.S = malloc(st.size * sizeof(char));
```

Correct:

```
st.size = strlen(exp);  
st.top = -1;  
st.S = malloc(st.size * sizeof(char));
```

Reason:

```
The stack must start empty.
```

Mistake 4: Popping without checking empty first

Wrong:

```
if (exp[i] == ')') {  
    pop(&st);  
}
```

Correct:

```
if (exp[i] == ')') {  
    if (isEmpty(st)) {  
        return 0;  
    }  
    pop(&st);  
}
```

Reason:

A closing parenthesis may appear without a matching opening parenthesis.

Mistake 5: Forgetting the final stack check

Wrong idea:

If the loop finishes, the expression must be balanced.

Correct idea:

After the loop, the stack must be empty.

Final check:

```
return isEmpty(st);
```

Mistake 6: Thinking balanced parentheses means valid expression

Example:

(A+)*(B)

Parentheses are balanced.

But the expression is mathematically invalid.

The algorithm only checks parentheses.

37. C++ Style Allocation Option

In C-style code, we used:

```
st.S = (char *)malloc(st.size * sizeof(char));
```

In C++, you could use:

```
st.S = new char[st.size];
```

And later release it using:

```
delete[] st.S;
```

Important rule:

Allocation method	Release method
<code>malloc()</code>	<code>free()</code>
<code>new[]</code>	<code>delete[]</code>

Do not mix them in beginner code.

38. Practice Inputs and Results

Try these expressions:

Expression	Result from algorithm	Reason
<code>(A+B)</code>	Balanced	One opening and one closing parenthesis.
<code>((A+B)</code>	Not Balanced	One opening parenthesis is left over.
<code>(A+B))</code>	Not Balanced	One extra closing parenthesis.
<code>((A+B)*(C-D))</code>	Balanced	All parentheses match correctly.
<code>A+B*C</code>	Balanced	No unmatched parentheses exist.

Expression	Result from algorithm	Reason
<code>(A+)*(B)</code>	Balanced	Parentheses match, but expression is mathematically invalid.
<code>)))</code>	Not Balanced	Closing parenthesis appears when stack is empty.
<code>((((</code>	Not Balanced	Opening parentheses remain at the end.

39. Practice Dry Run: `((A+B)*(C-D))`

Expression:

`((A+B)*(C-D))`

Only parentheses matter:

`( ( ) ( ) )`

Trace:

Step	Character group	Action	Stack after action
1	<code>(</code>	push	<code>(</code>
2	<code>(</code>	push	<code>((</code>
3	<code>A+B</code>	ignore	<code>((</code>
4	<code>)</code>	pop	<code>(</code>
5	<code>*</code>	ignore	<code>(</code>
6	<code>(</code>	push	<code>((</code>
7	<code>C-D</code>	ignore	<code>((</code>
8	<code>)</code>	pop	<code>(</code>
9	<code>)</code>	pop	empty

Final stack:

empty

Result:

Balanced

40. Lesson 10 Summary

Lesson 10 teaches how to implement parenthesis matching using an array stack that stores characters.

The most important ideas are:

- Parentheses are characters, so the stack uses `char *S`.
- The stack still uses `size` and `top`.
- Empty condition is still `top == -1`.
- Full condition is still `top == size - 1`.
- `strlen(exp)` gives a safe stack size.
- The string is scanned until `\0`.
- Push every opening parenthesis `(`.
- For every closing parenthesis `)`, pop one opening parenthesis.
- If a closing parenthesis appears while the stack is empty, the expression is not balanced.
- At the end, the stack must be empty.
- Time complexity is $O(n)$.
- Space complexity is $O(n)$.

The core sentence to remember is:

Use a character stack. Push every opening parenthesis. For every closing parenthesis, pop one opening parenthesis. If you cannot pop, it is unbalanced. At the end, the stack must be empty.

41. Quick Self-Check

Answer these without looking at the code:

1. Why does Lesson 10 use `char *S` instead of `int *S`?
2. What does `top = -1` mean?
3. Why is `strlen(exp)` used as the stack size?
4. What does `\0` mean in a C string?

5. What action happens when the algorithm sees `(`?
 6. What action happens when the algorithm sees `)`?
 7. Why must we check `isEmpty()` before `pop()`?
 8. What does it mean if the stack is not empty at the end?
 9. Does this algorithm check whether the full expression is mathematically valid?
 10. What are the time and space complexities?
-

42. Final Beginner Explanation

Think of every `(` as a task that says:

I need to be closed later.

So we push it onto the stack.

Think of every `)` as saying:

I will close the most recent unmatched opening parenthesis.

So we pop one `(` from the stack.

If a `)` appears and the stack is empty, there is nothing to close.

That means the expression is not balanced.

If the scan ends and the stack still has `(` left, those parentheses were never closed.

That also means the expression is not balanced.

If the scan ends and the stack is empty, every opening parenthesis found a matching closing parenthesis.

That means the expression is balanced.